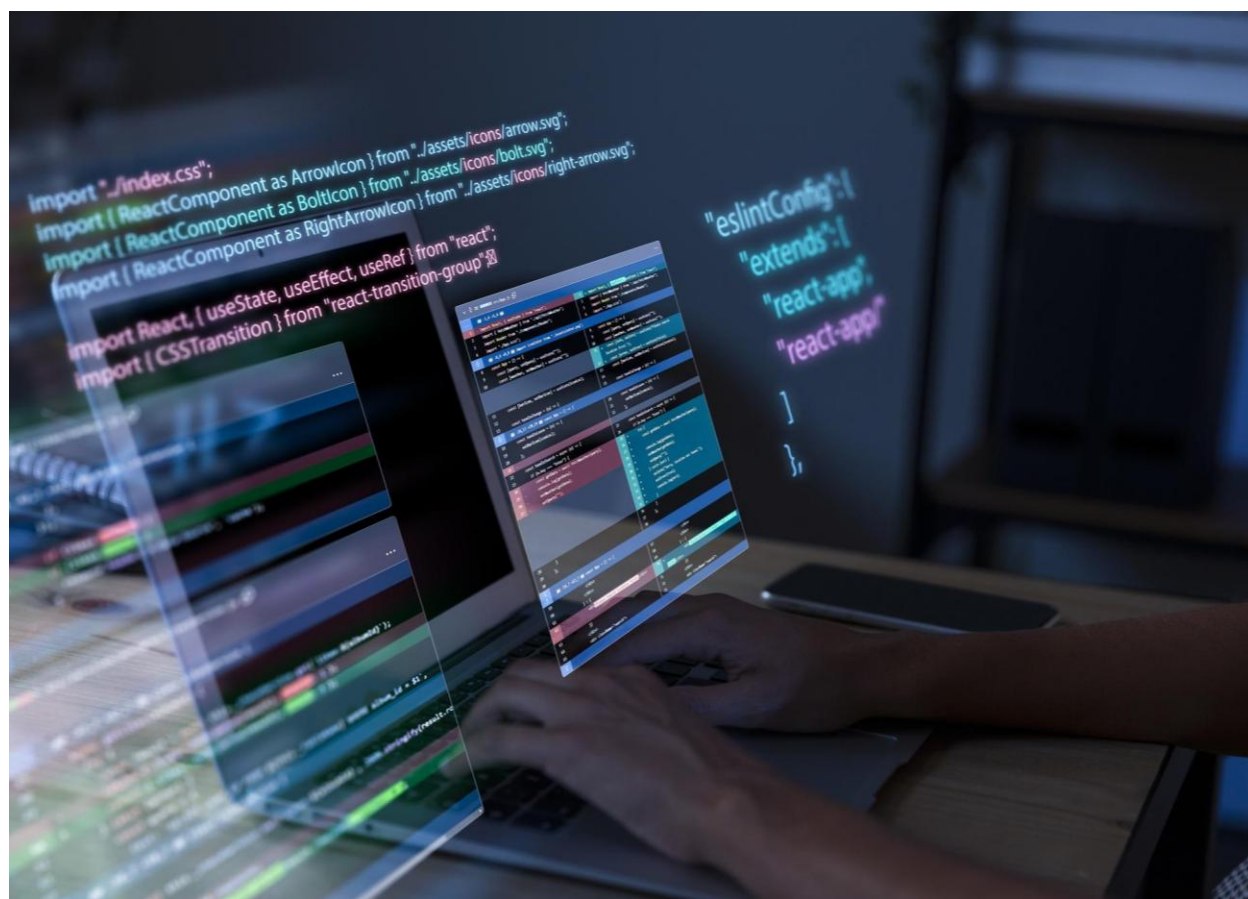


Ejercicios

Colecciones (III)



Autor: Marcela Martín

Fecha: abril 2026

1. Desarrolla una aplicación para gestionar una colección de empleados, permitiendo la manipulación de sus salarios mediante un iterador y la gestión de errores específicos/personalizados.

a) Clase **Empleado**, incluye:

- nombre (String) para almacenar el nombre del empleado.
- edad (int) para almacenar la edad del empleado.
- salario (double) para almacenar el salario del empleado.
- constructor que inicialice los 3 atributos.
- getters/setters
- toString().

b) Excepción Verificada Personalizada **EmpleadoNoEncontradoException**:

- Incluye un constructor que pueda recibir un mensaje descriptivo de la excepción.

c) Clase **GestionEmpleados**, contiene:

- colección para almacenar objetos de tipo Empleado (ArrayList)
- método público *agregarEmpleado(Empleado empleado)* que permita añadir un objeto Empleado a la lista.
- método público llamado *obtenerIteradorEmpleados()* que devuelva un objeto de tipo java.util.Iterator para recorrer la lista de empleados.
- método público *aumentarSalarioEmpMayoresDe30(double porcentajeAumento)* que realice las siguientes acciones:
 - Obtener un Iterator de la lista de empleados utilizando el método *obtenerIteradorEmpleados()*.
 - Itera sobre la lista de empleados utilizando el Iterator.
 - Por cada empleado en la lista, verifica si su edad es mayor que 30.

- Si la edad del empleado es mayor que 30, aumenta su salario en el porcentajeAumento especificado.
 - método público *buscarEmpleadoPorNombre(String nombre)* que recorre la lista de empleados y:
 - Si encuentra un empleado cuyo nombre coincida con el nombre proporcionado, devuelve ese objeto Empleado.
 - Si después de recorrer toda la lista no se encuentra ningún empleado con el nombre especificado, lanza una excepción de tipo *EmpleadoNoEncontradoException* con un mensaje informativo (por ejemplo, "No se encontró ningún empleado con el nombre: [nombre]").
- d) Clase **AppEmpleados**, para probar la correcta implementación del sistema:
- Crea varias instancias de la clase Empleado y las agrega a la instancia de GestionEmpleados.
 - Llama al método *aumentarSalarioEmpleadosMayoresDe30*.
 - Intenta buscar un empleado por su nombre utilizando el método *buscarEmpleadoPorNombre*.
 - Implementa el correcto tratamiento de excepciones.
2. Desarrolla una aplicación para gestionar una Biblioteca (colección de libros), utilizando un iterador para procesar los libros que cumplan con un criterio específico y manejando condiciones de datos inválidos mediante una excepción personalizada.
- a) Clase Excepción Verificada Personalizada **LibroInvalidoException**:
- Incluye un constructor público para esta excepción que pueda recibir un mensaje descriptivo del error.

b) Clase **Libro**, incluye:

- titulo (String) para almacenar el título del libro.
- autor (String) para almacenar el nombre del autor del libro.
- anyoPublicacion (int) para almacenar el año de publicación del libro.
- constructor que inicialice los 3 atributos.
- getters/setters.
- toString() para devolver una representación legible del objeto Libro, incluyendo todos sus atributos.
- método *validarLibro()*:
 - Verifica si el atributo autor del libro es null o está vacío (después de eliminar espacios en blanco al inicio y al final).
 - Si cualquiera de estas condiciones se cumple para un libro, lanza una instancia de la excepción LibroInvalidoException con un mensaje descriptivo que indique qué atributo (título o autor) es inválido.

c) Clase **GestionBiblioteca**, incluye:

- Colección para almacenar Libros (ArrayList)
- método *agregarLibro(Libro libro)* que añade el libro al ArrayList si este se valida correctamente.
- método público llamado *imprimirLibrosPosterioresA(int anyo)* que:
 - crea un Iterator para la lista de libros.
 - Utiliza el iterador para recorrer la lista de libros.
 - Realiza una validación del libro actual con el método *validarLibro(Libro libro)* que lanza la excepción LibroInvalidoException.
 - Si el libro no es válido, no se imprime

- Por cada libro válido, verifica si su `anyoPublicacion` es estrictamente mayor que el año proporcionado como argumento al método.
- Si el año de publicación cumple con la condición, imprime la información del libro utilizando su método `toString()`.

d) Clase **AppBiblio** para probar la correcta implementación del sistema:

- Crea varias instancias de la clase `Libro`, incluyendo algunos libros con títulos o autores nulos o vacíos para probar la funcionalidad de la excepción.
- Agrega estas instancias a la lista de libros.
- Llama al método `imprimirLibrosPosterioresA()` con un año específico para filtrar e imprimir los libros.
- Implementa el correcto tratamiento de excepciones.